

Air University



Digital Logic Design (Lab)

Year 2025

Ms. Faryal Naeem

Academic Year: 2024 – 2028

Department of Cyber Security

USAMA ARSHAD | 247141

BS – Cyber Security – Fall – 24 (A)

Date of Submission: May 16, 2025

Experiment No. 11

Title: Flip Flops

Objectives

For SR Latch:

- Understand the operational principle of the SR Latch as a basic sequential circuit.
- Construct an SR Latch utilizing NAND gates.
- Verify the functionality of the SR Latch by examining its truth table.
- Analyze the feedback mechanism within a sequential circuit.

For SR Flip-Flop (Clocked):

- Distinguish between a latch and a flip-flop.
- Create a clocked SR Flip-Flop using NAND gates.
- Validate the synchronous operation of the flip-flop by studying its truth table.
- Explore how clock signals influence state changes in sequential circuits.

Equipment and Components

- Proteus Software (for simulation)
- Logic Trainer Kit (for practical application)
- NAND Gate ICs (7400 series)
- Manual input switches or wires (for S, R, and clock inputs)
- LEDs (to indicate output)
- Resistors (to limit LED current)
- Breadboard and connecting wires

Introduction

Sequential circuits form the foundation of digital memory systems, as their outputs depend on both current inputs and historical states. This distinct feedback mechanism allows for data storage, which differentiates sequential circuits from combinational circuits. The SR (Set-Reset) Latch, constructed with cross-coupled NAND gates, is a vital digital memory element that retains a single bit of data without the need for clock synchronization. It changes state based on the Set (S) and Reset (R) inputs.

The SR Flip-Flop extends this concept by incorporating clock control, thus functioning as a synchronized memory element. Unlike the latch, the flip-flop only modifies its state when the clock signal is active, making synchronization essential for dependable sequential logic in digital systems such as registers and counters. This experiment examines both circuits, highlighting their behavior, truth tables, and roles in digital memory applications.

Procedure

A. Proteus Software Implementation

a. SR Latch:

Software Setup:

- Launch the Proteus simulation software.
- Start a new project.

Circuit Design:

- Choose and position two NAND gates from the component library.
- Establish cross-coupling by linking the output of the first NAND gate to an input of the second NAND gate.
- Connect the output of the second NAND gate back to an input of the first NAND gate.
- Add logic switches for the S and R inputs.
- Incorporate logic probes to observe the outputs Q and $\bar{Q}$.

Input Configuration:

- Include two logic state components for the S and R inputs.
- Connect these components to the available inputs of the NAND gates.

Output Monitoring:

- Add logic probes to the outputs Q and $\bar{Q}$.
- Clearly label all components and connections for clarity.

Simulation:

- Begin the simulation and test various combinations of S and R inputs:
 - S=0, R=0 (Invalid state)
 - S=0, R=1 (Set Q=1)
 - S=1, R=0 (Reset Q=0)
 - S=1, R=1 (Hold/No Change)
- Observe and document the output states corresponding to each input combination.

b. SR Flip-Flop (Clocked):

Circuit Design:

- Use four NAND gates from the component library for the design.
- Set up two NAND gates as input gates, including S, R, and clock inputs.
- Arrange the other two NAND gates in a cross-coupling configuration (similar to the SR Latch).
- Connect the outputs of the input gates to the cross-coupled NAND gates.

Input Configuration:

- Add logic switches for S, R, and clock inputs.
- Wire these to the input NAND gates as indicated in the schematic.

Simulation:

- Run the simulation with the clock line set to 0 and toggle the S and R inputs to confirm the output remains unchanged.
- Set the clock line to 1 and test various combinations of S and R inputs:
 - S=0, R=0 (Hold/No Change)
 - S=0, R=1 (Reset Q=0)
 - S=1, R=0 (Set Q=1)
 - S=1, R=1 (Invalid state)
- Monitor for outputs that only change when the clock is high.

B. Logic Trainer Implementation**a. SR Latch:****Hardware Setup:**

- Start by placing a 7400 IC onto the breadboard.
- Interconnect two NAND gates by linking pin 1 with pin 3 and pin 4 with pin 6.
- Establish connections from manual switches to the Set (S) input (pin 1) and the Reset (R) input (pin 4).
- Connect LEDs, each paired with a resistor, to the Q output (pin 3) and the $\bar{Q}$ output (pin 6).

Testing:

- Activate the circuit with a 5V power supply.
- Flip the S and R switches to observe the LED states.
- Ensure the performance aligns with the outcomes you noticed in simulation software such as Proteus.

b. SR Flip-Flop (Clocked):**Hardware Setup:**

- Utilize two 7400 ICs to provide a total of four NAND gates.
- Attach a manual switch to the clock input, which will manage the Set (S) and Reset (R) inputs via the input NAND gates.
- Cross-connect the output NAND gates to set up feedback.
- Connect LEDs to the Q and $\bar{Q}$ outputs.

Testing:

- Begin with the Clock in the low position (switch off) and alternate the S and R inputs; the output should remain unchanged.
- When the Clock is set to high (switch on), evaluate the inputs:
 - If S = 1 and R = 0, Q should show an output of 1.
 - If S = 0 and R = 1, Q should display an output of 0.

Truth Tables

1. SR Latch:

S	R	$Q_{(n+1)}$	Description
0	0	Invalid	Forbidden state
0	1	1	Set
1	0	0	Reset
1	1	$Q_{(n)}$	Hold / No Change

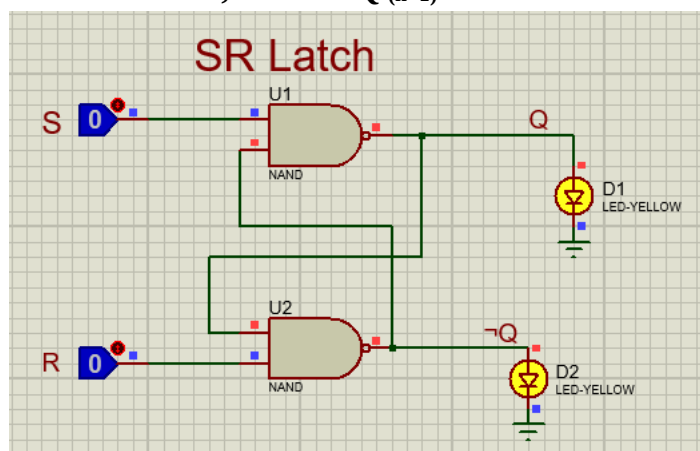
2. SR Flip Flop (Clocked):

Clock	S	R	$Q_{(n+1)}$	Description
1	0	0	$Q_{(n)}$	Hold
1	0	1	0	Reset
1	1	0	1	Set
1	1	1	Invalid	Forbidden state

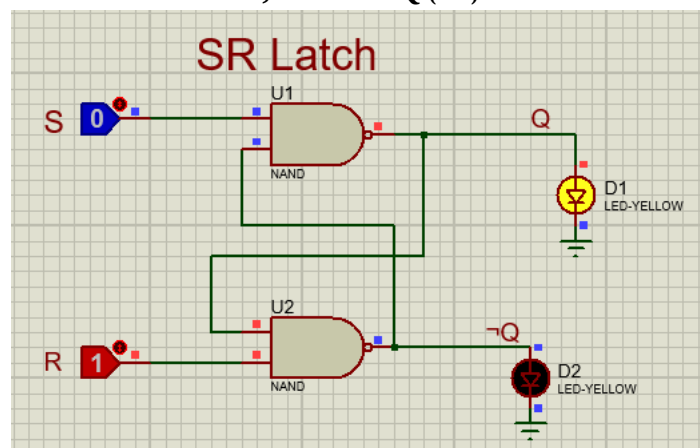
Simulated Results

1. SR Latch:

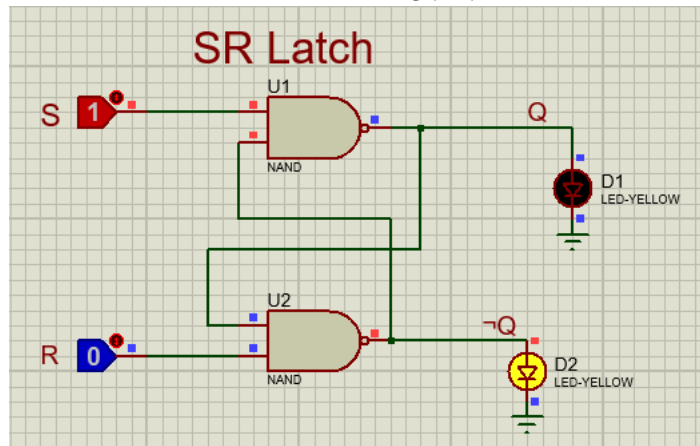
$$S = 0, R = 0 \Rightarrow Q_{(n+1)} = \text{Invalid}$$



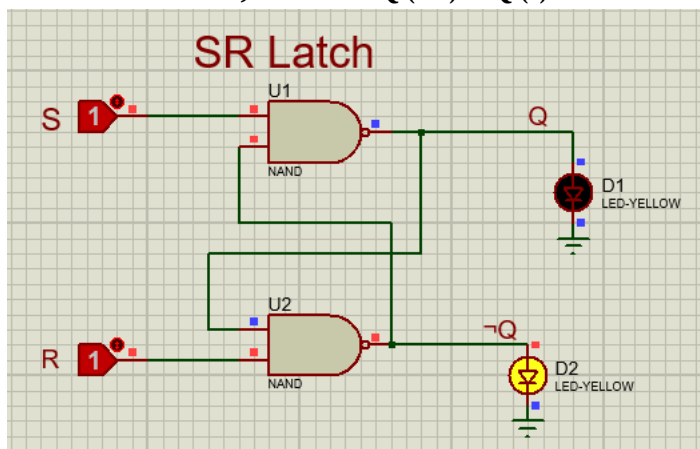
$$S = 0, R = 1 \Rightarrow Q_{(n+1)} = 1$$



$$S = 1, R = 0 \Rightarrow Q_{(n+1)} = 0$$

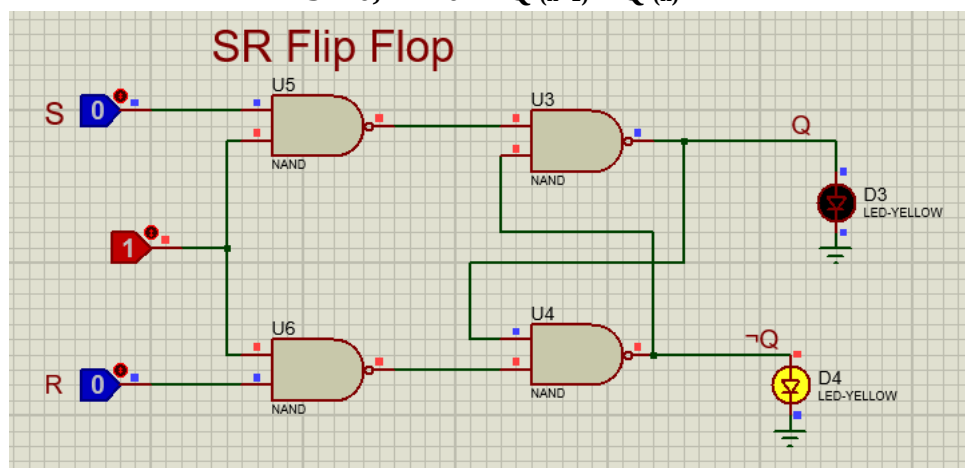


$$S = 1, R = 1 \Rightarrow Q_{(n+1)} = Q_{(n)}$$

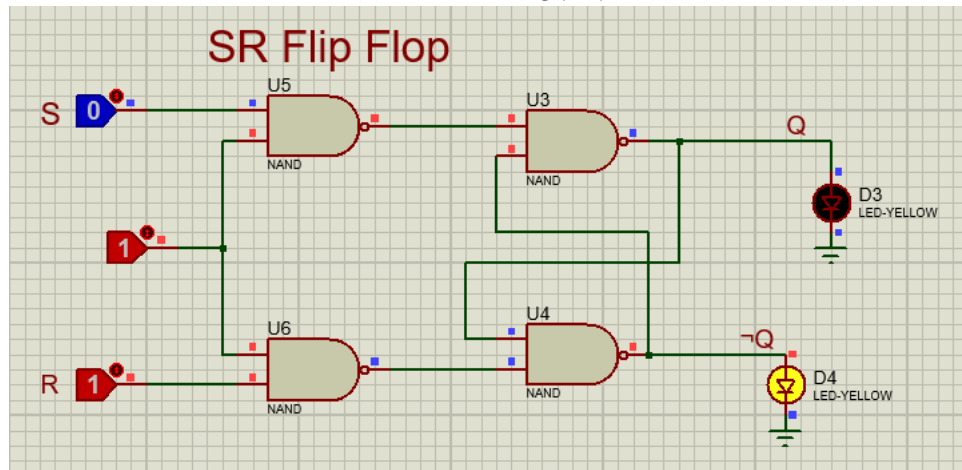


2. SR Flip Flop (Clocked):

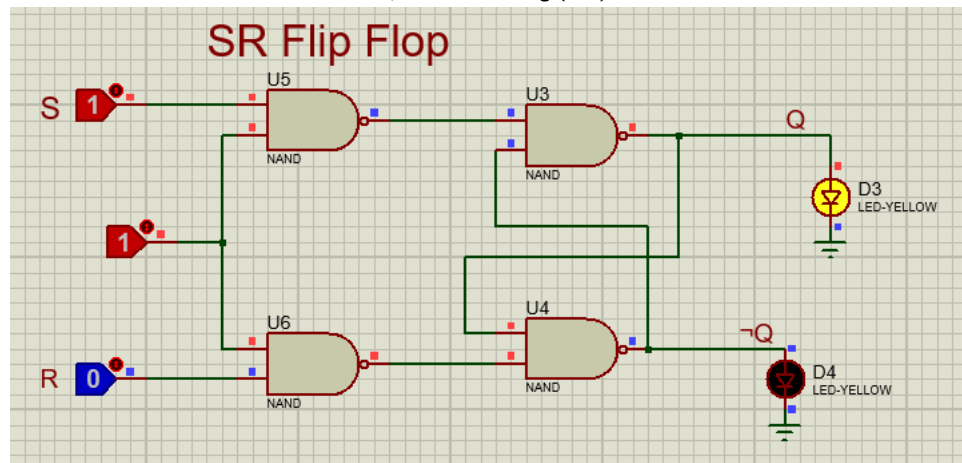
$$S = 0, R = 0 \Rightarrow Q_{(n+1)} = Q_{(n)}$$



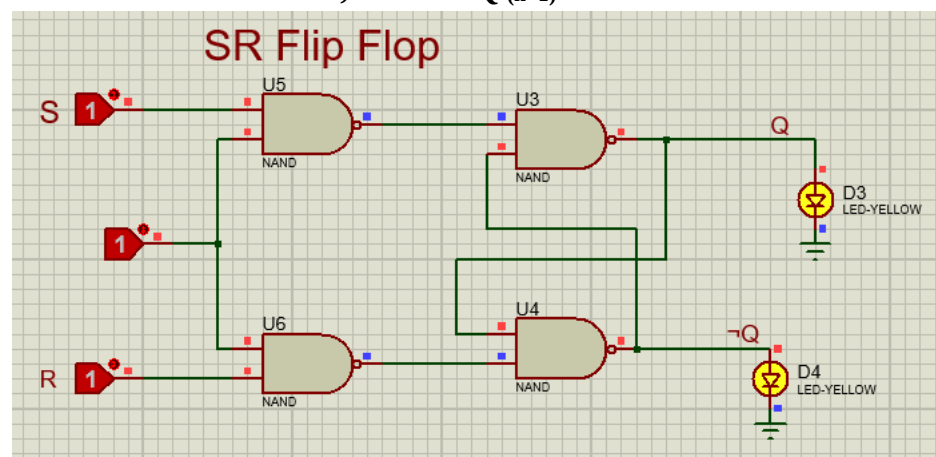
$$S = 0, R = 1 \Rightarrow Q_{(n+1)} = 0$$



$$S = 1, R = 0 \Rightarrow Q_{(n+1)} = 1$$



$$S = 1, R = 1 \Rightarrow Q_{(n+1)} = \text{Invalid}$$



Working Principle:

SR Latch:

The SR Latch operates through feedback, utilizing two linked NAND gates to form a bistable circuit. The output of each gate is connected back to the input of the other, resulting in a memory effect. When the inputs are set to $S = 0$ and $R = 1$, the output Q is set to 1. In contrast, with $S = 1$ and $R = 0$, the Q output is reset to

0. If both S and R are 1, the latch retains its previous state. The condition where $S = R = 0$ is deemed an invalid state and is generally not used in standard operations.

SR Flip-Flop (Clocked):

The SR Flip-Flop expands on the SR Latch's features by adding clock synchronization. The clock input dictates when the flip-flop will react to the S and R inputs. When the clock is low (0), the flip-flop disregards these inputs, keeping its current state. When the clock transitions to high (1), the flip-flop functions like an SR Latch. This clock-controlled aspect is essential for synchronizing multiple flip-flops within sequential circuits.

Applications:

- Memory cells in digital systems
- Registers and data storage
- Timing and synchronization circuits
- Debouncing switches
- State machines in digital control systems

Conclusions

This experiment demonstrated how the SR Latch and SR Flip-Flop can be constructed and their behaviors using NAND gates in a Proteus simulation. The SR Latch exhibited asynchronous memory behavior, altering states instantly in response to input signals. In contrast, the SR Flip-Flop showcased synchronous operation, with state changes regulated by the clock signal. Both circuits effectively illustrated the principles of feedback and memory in sequential logic.

The truth tables validated the anticipated behaviors of both circuits, outlining the set, reset, and hold states. Understanding these basic components is crucial for creating more intricate sequential circuits, such as counters, registers, and state machines.

THE END
